

‘Bases de Datos NO SQL’ Evaluación

Alumno: David Carrera Pintor

Profesor: Álvaro Bravo

use Movies

// 1. Analizar con find la colección.

db.movies.find()

```
ovies | 0.037 s | 28.795 Docs
/* 1 createdAt:24/4/2025 18:20:32*/
{
  "_id" : ObjectId("680a64d067f3035c2444acfe"),
  "title" : "Caught",
  "year" : 1900,
  "cast" : [ ],
  "genres" : [ ]
},
/* 2 createdAt:24/4/2025 18:20:32*/
{
  "_id" : ObjectId("680a64d067f3035c2444acff"),
  "title" : "After Dark in Central Park",
  "year" : 1900,
  "cast" : [ ],
  "genres" : [ ]
},
/* 3 createdAt:24/4/2025 18:20:32*/
{
  "_id" : ObjectId("680a64d067f3035c2444ad00"),
  "title" : "Buffalo Bill's Wild West Parad",
  "year" : 1900,
  "cast" : [ ],
  "genres" : [ ]
}
```

// 2. Contar cuántos documentos (películas) tiene cargado.

db.movies.find().count()

```
12
Find x Result x
0.047 s
1 28795
```

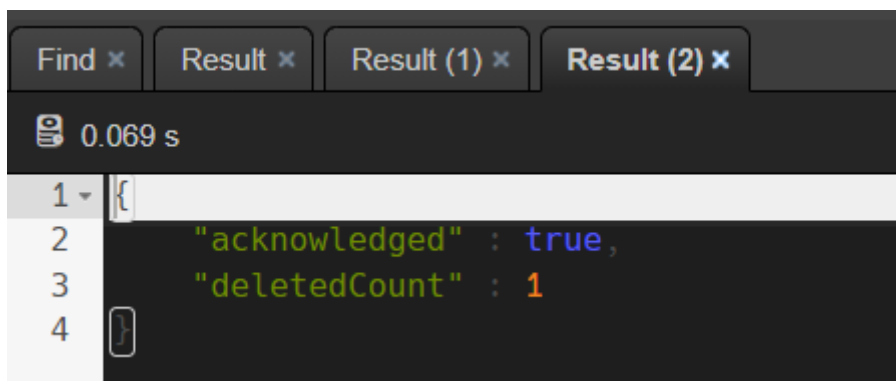
// 3. Insertar una película.

```
var nuevaPelicula = {title: "Nueva Pelicula", year: 1999, cast: ["David"], genres: ["Drama"]}
db.movies.insertOne(nuevaPelicula)
```



// 4. Borrar la película insertada en el punto anterior (en el 3).

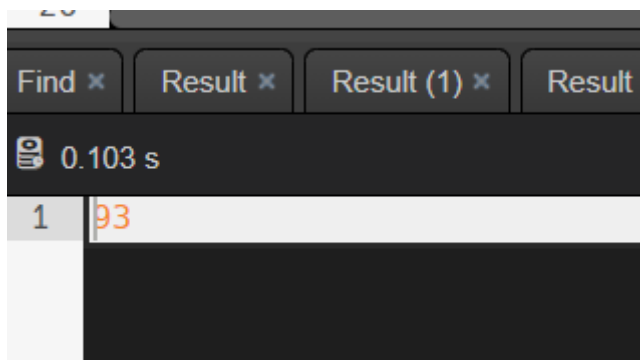
```
db.movies.deleteOne({title: "Nueva Pelicula"})
```



// 5. Contar cuantas películas tienen actores (cast) que se llaman "and".

// Estos nombres de actores están por ERROR.

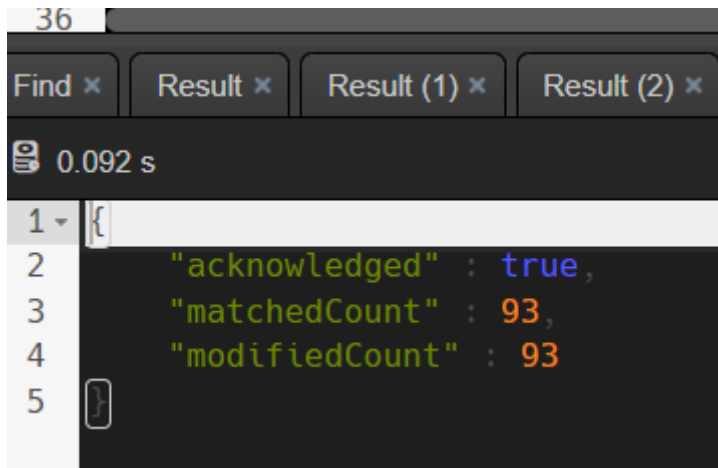
```
db.movies.find({cast: "and"}).count()
```



```
// 6. Actualizar los documentos cuyo actor (cast) tenga por error el valor "and" como si
// realmente fuera un actor. Para ello, se debe sacar únicamente ese valor del array cast.
// Por lo tanto, no se debe eliminar ni el documento (película) ni su array cast con el resto de
// actores.
```

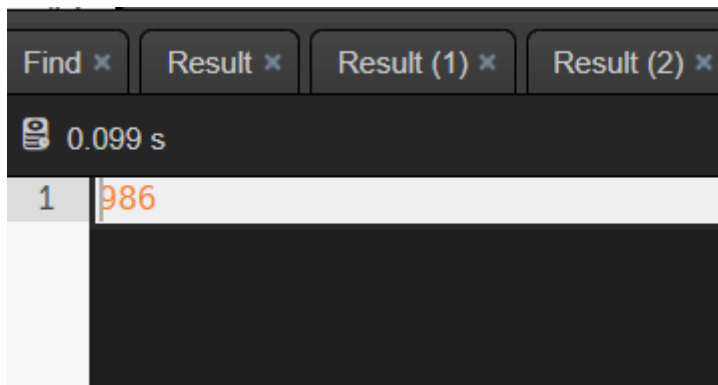
```
var filtro = {cast: "and"};
var eliminar = {$pull: {cast: "and"}}

db.movies.updateMany(filtro, eliminar)
```



```
// 7. Contar cuantos documentos (películas) tienen el array 'cast' vacío.

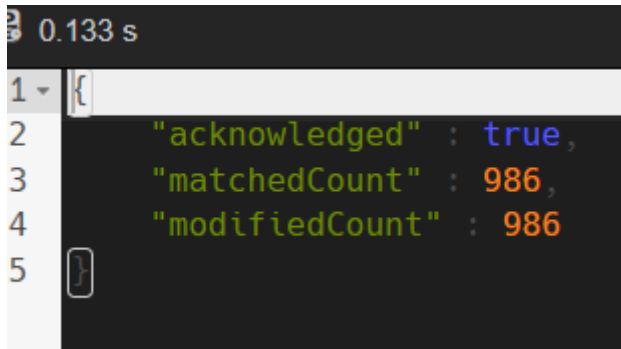
db.movies.find({"cast": {$size: 0}}).count()
```



// 8. Actualizar TODOS los documentos (películas) que tengan el array cast vacío, añadiendo
// un nuevo elemento dentro del array con valor Undefined. Cuidado! El tipo de cast debe seguir
// siendo un array. El array debe ser así -> ["Undefined"].

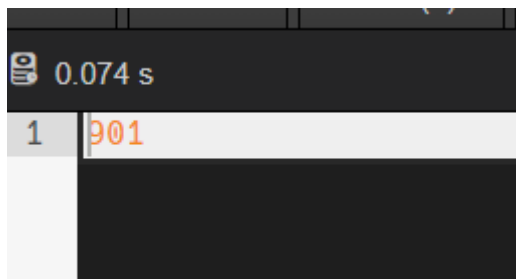
```
var filtro2 = {"cast": {$size: 0}}  
var actualizar = {$set: { "cast": ["Undefined"]}}
```

```
db.movies.updateMany(filtro2, actualizar)
```



// 9. Contar cuantos documentos (películas) tienen el array genres vacío.

```
db.movies.find({"genres": {$size: 0}}).count()
```



// 10. Actualizar TODOS los documentos (películas) que tengan el array genres vacío,
// añadiendo un nuevo elemento dentro del array con valor Undefined. Cuidado! El tipo de
// genres debe seguir siendo un array. El array debe ser así -> ["Undefined"]

```
var filtro3 = {"genres": {$size: 0}}  
var actualizar2 = { $set: {"genres": ["Undefined"]}}
```

```
db.movies.updateMany(filtro3, actualizar2)
```

```
db.movies.find({"genres": "Undefined"})
```

```

56
Find x Result x Result (1) x Result (2) x Result (3) x Result (4) x Result (5) x Re
movies 0.112 s 901 Docs
1 /* 1 createdAt:24/4/2025 18:20:32*/
2 {
3   "_id" : ObjectId("680a64d067f3035c2444acfe"),
4   "title" : "Caught",
5   "year" : 1900,
6   "cast" : [ "Undefined" ],
7   "genres" : [ "Undefined" ]
8 },
9
10 /* 2 createdAt:24/4/2025 18:20:32*/
11 {
12   "_id" : ObjectId("680a64d067f3035c2444acff"),
13   "title" : "After Dark in Central Park",
14   "year" : 1900,
15   "cast" : [ "Undefined" ],
16   "genres" : [ "Undefined" ]
17 },
18
19 /* 3 createdAt:24/4/2025 18:20:32*/
20 {
21   "_id" : ObjectId("680a64d067f3035c2444ad00"),
22   "title" : "Buffalo Bill's Wild West Parad",
23   "year" : 1900,

```

// 11. Mostrar el año más reciente / actual que tenemos sobre todas las películas.

```
db.movies.find({}, {"year" : 1, "_id" : 0}).sort({"year": -1 }).limit(1)
```

```

movies 0.099 s 1 Doc
1 {
2   "year" : 2018
3 }

```

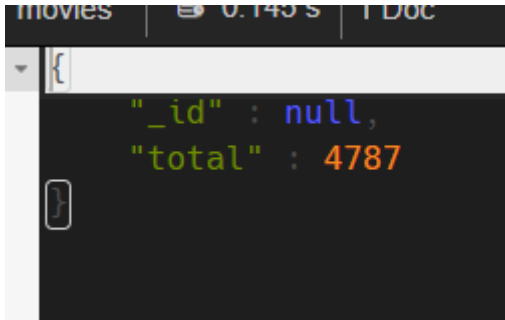
// 12. Contar cuántas películas han salido en los últimos 20 años. Debe hacerse desde el último año que se tienen registradas películas en la colección, no desde la fecha actual, mostrando el resultado total de esos años, no el resultado por año. Revisar que no se cuentan 21 años. // Se debe hacer con el Framework de Agregación.

```
var annoMaximo = db.movies.find({}, {year: 1, _id: 0}).sort({year: -1}).limit(1).toArray()[0].year;
var annoMinimo = annoMaximo - 20;
```

```
var filtroAnnos = { year: { $gt: annoMinimo, $lte: annoMaximo } };
```

```
var resultado = db.movies.aggregate([
  { $match: filtroAnnos },
  { $group: { _id: null, total: { $sum: 1 } } }
]);
```

resultado;



// 13. Contar cuántas películas han salido en la década de los 60 (del 60 al 69 incluidos). Se
// debe hacer con el Framework de Agregación.

```
var filtroAnnos = { year: { $gte: 1960, $lte: 1969 } };  
  
db.movies.aggregate([  
  {$match: filtroAnnos},  
  {$group: { _id: null, total: { $sum: 1 } }}  
]);
```



// 14. Mostrar el año u años con más películas mostrando el número de películas de ese año.
// Revisar si varios años pueden compartir tener el mayor número de películas.

```
var maxPelis = db.movies.aggregate([  
  {$group: { _id: "$year", pelis: { $sum: 1 } }},  
  {$sort: { pelis: -1 }},  
  {$group: { _id: "$pelis", years: { $push: "$_id" } }},  
  {$sort: { _id: -1 }},  
  {$limit: 1},  
  {$project: { _id: 0, pelis: "$_id", years: 1 } }  
]).toArray();  
  
maxPelis;
```

```

1 {
2   "years" : [ 1919 ],
3   "pelis" : 634
4 }

```

// 15. Mostrar el año u años con menos películas mostrando el número de películas de ese año.
 // Revisar si varios años pueden compartir tener el menor número de películas.

```

var minPelis = db.movies.aggregate([
  { $group: { _id: "$year", pelis: { $sum: 1 } } },
  { $sort: { pelis: 1 } },
  { $group: { _id: "$pelis", years: { $push: "$_id" } } },
  { $sort: { _id: 1 } },
  { $limit: 1 },
  { $project: { _id: 0, pelis: "$_id", years: 1 } }
]).toArray();

```

minPelis;

```

1 {
2   "years" : [ 1907, 1906, 1902 ],
3   "pelis" : 7
4 }

```

// 16. Guardar en nueva colección llamada "actors" realizando la fase \$unwind por cast, no se
 // debe agrupar de nuevo y debe contener todas las claves (title,year,cast,genres).

```

db.movies.aggregate([
  { $unwind: "$cast" },
  { $project: { _id: 0 } },
  { $out: "actors" }
]);

```

db.actors.find().count()

```

0.832 s
1 33224

```


// 17. Sobre actores (nueva colección), mostrar la lista con los 5 actores que han participado en
// más películas mostrando el número de películas en las que ha participado. Importante! Se
// necesita previamente filtrar para descartar aquellos actores llamados "Undefined".

```
db.actors.aggregate([
  { $match: { cast: { $ne: "Undefined" } } },
  { $group: { _id: "$cast", cuenta: { $sum: 1 } } },
  { $sort: { cuenta: -1 } },
  { $limit: 5 },
  { $project: { _id: 1, cuenta: 1 } }
]);
```

```
/* 1 */
{
  "_id" : "Harold Lloyd",
  "cuenta" : 190
},
/* 2 */
{
  "_id" : "Hoot Gibson",
  "cuenta" : 142
},
/* 3 */
{
  "_id" : "John Wayne",
  "cuenta" : 136
},
/* 4 */
{
  "_id" : "Charles Starrett",
  "cuenta" : 116
},
/* 5 */
{
  "_id" : "Bebe Daniels",
  "cuenta" : 103
}
```

// 18. Sobre actores (nueva colección), agrupar por película y año mostrando las 5 en las que
// más actores hayan participado, mostrando el número total de actores.

```
db.actors.aggregate([
  { $group: { _id: { title: "$title", year: "$year" }, actores: { $addToSet: "$cast" } } },
  { $project: { _id: 1, cuenta: { $size: "$actores" } } },
  { $sort: { cuenta: -1 } },
  { $limit: 5 }
]);
```

```
actors | 0.519 s | 5 Docs
1 /* 1 */
2 {
3   "_id" : {
4     "title" : "The Twilight Saga: Breaking Dawn - Part 2",
5     "year" : 2012
6   },
7   "cuenta" : 35
8 },
9
10 /* 2 */
11 {
12   "_id" : {
13     "title" : "Anchorman 2: The Legend Continues",
14     "year" : 2013
15   },
16   "cuenta" : 33
17 },
18
19 /* 3 */
20 {
21   "_id" : {
22     "title" : "Cars 2",
23     "year" : 2011
24   },
25 }
```

// 19. Sobre actores (nueva colección), mostrar los 5 actores cuya carrera haya sido la más
// larga. Para ello, se debe mostrar cuándo comenzó su carrera, cuándo finalizó y cuántos años
// ha trabajado. Importante! Se necesita previamente filtrar para descartar aquellos actores
// llamados "Undefined". Aclarar que no se eliminan de la colección, sólo que filtramos para que
// no aparezcan.

```
db.actors.aggregate([
  { $match: { cast: { $ne: "Undefined" } } },
  { $group: { _id: "$cast", comienza: { $min: "$year" }, termina: { $max: "$year" } } },
  { $project: { comienza: 1, termina: 1, annos: { $add: [ { $subtract: ["$termina", "$comienza"] }, 1 ] } } },
  { $sort: { annos: -1 } },
  { $limit: 5 }
]);
```

```
/* 1 */
{
  "_id" : "Harrison Ford",
  "comienza" : 1919,
  "termina" : 2017,
  "annos" : 99
},

/* 2 */
{
  "_id" : "Gloria Stuart",
  "comienza" : 1932,
  "termina" : 2012,
  "annos" : 81
},

/* 3 */
{
  "_id" : "Kenny Baker",
  "comienza" : 1937,
```

// 20. Sobre actors (nueva colección), Guardar en nueva colección llamada “genres” realizando la fase \$unwind por genres, no se debe agrupar de nuevo y debe contener todas las claves (title,year,cast,genres). Después, contar cuantos documentos existen en la nueva colección.

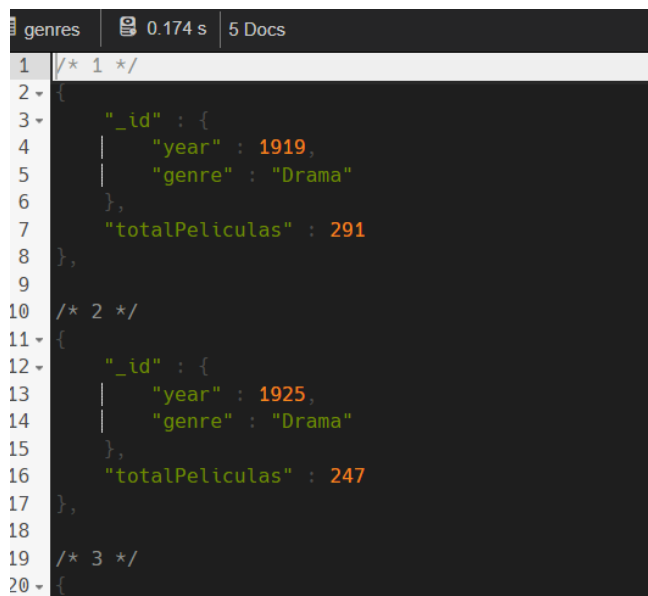
```
db.actors.aggregate([
  { $unwind: "$genres" },
  { $project: { _id: 0 } },
  { $out: "genres" }
]);
```

```
db.genres.find().count()
```

```
1.044 s
104950
```

// 21. Sobre genres (nueva colección), mostrar los 5 documentos agrupados por "Año y Género" que más número de películas diferentes tienen mostrando el número total de películas. Importante! Se necesita previamente filtrar para descartar aquellos genres llamados "Undefined". Aclarar que no se eliminan de la colección, sólo que filtramos para que no aparezcan.

```
db.genres.aggregate([
  { $match: { genres: { $ne: "Undefined" } } },
  { $group: { _id: { year: "$year", genre: "$genres" }, peliculas: { $addToSet: "$title" } } },
  { $project: { _id: 1, totalPeliculas: { $size: "$peliculas" } } },
  { $sort: { totalPeliculas: -1 } },
  { $limit: 5 }
]);
```



```
1 /* 1 */
2 {
3   "_id" : {
4     "year" : 1919,
5     "genre" : "Drama"
6   },
7   "totalPeliculas" : 291
8 },
9
10 /* 2 */
11 {
12   "_id" : {
13     "year" : 1925,
14     "genre" : "Drama"
15   },
16   "totalPeliculas" : 247
17 },
18
19 /* 3 */
20 {
```

// 22. Sobre genres (nueva colección), mostrar los 5 actores y los géneros en los que han participado con más número de géneros diferentes, se debe mostrar tanto el número de géneros diferentes que ha interpretado como los diferentes géneros además del nombre del actor. Importante! Se necesita previamente filtrar para descartar aquellos actores llamados "Undefined".

```
db.genres.aggregate([
  { $match: { cast: { $ne: "Undefined" }, genres: { $ne: "Undefined" } } },
  { $group: { _id: "$cast", generos: { $addToSet: "$genres" } } },
  { $project: { _id: 1, numgeneros: { $size: "$generos" }, generos: 1 } },
  { $sort: { numgeneros: -1 } },
  { $limit: 5 }
]);
```

```

genres | 0.291 s | 5 Docs
/* 1 */
{
  "_id" : "Dennis Quaid",
  "generos" : [
    "Science Fiction",
    "Biography",
    "Crime",
    "Sports",
    "Horror",
    "Family",
    "Suspense",
    "Romance",
    "Dance",
    "Disaster",
    "Animated",
    "Thriller",
    "Adventure",
    "Satire",
    "Musical",
    "Western",
    "Action",
    "Fantasy",
    "Drama",
    "Comedy"
  ],
  "numgeneros" : 20
},
/* 2 */
{
  "_id" : "Danny Glover",
  "generos" : [

```

// 23. Sobre genres (nueva colección), mostrar las 5 películas y su año correspondiente en los que más géneros diferentes han sido catalogados, mostrando esos géneros y el número de géneros que contiene. Importante! Se necesita previamente filtrar para descartar aquellos genres llamados "Undefined".

```

db.genres.aggregate([
  { $match: { genres: { $ne: "Undefined" } } },
  { $group: { _id: { title: "$title", year: "$year" }, genres: { $addToSet: "$genres" } } },
  { $project: { numgeneros: { $size: "$genres" }, genres: 1 } },
  { $sort: { numgeneros: -1 } },
  { $limit: 5 }
]);

```

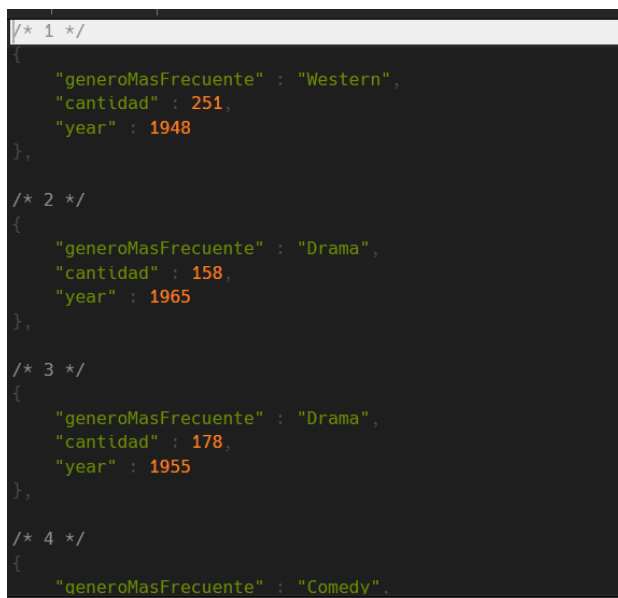
```

genres | 0.280 s | 5 Docs
/* 1 */
{
  "_id" : {
    "title" : "American Made",
    "year" : 2017
  },
  "generos" : [
    "Thriller",
    "Biography",
    "Historical",
    "Crime",
    "Comedy",
    "Drama",
    "Action"
  ],
  "numgeneros" : 7
},
/* 2 */
{
  "_id" : {
    "title" : "My Little Pony: The Movie",
    "year" : 2017
  },
  "generos" : [
    "Animated",
    "Fantasy",
    "Musical",
    "Adventure",
    "Family",
    "Comedy"
  ],
  "numgeneros" : 6
},

```

// 24 Sobre genres (nueva colección), encontrar el género más frecuente en películas de cada año, mostrando el año, el género y cuántas veces aparece como el más repetido en ese año. Se filtra para descartar aquellos actores llamados "Undefined".

```
db.genres.aggregate([
  { $match: { genres: { $ne: "Undefined" } } },
  { $group: { _id: { year: "$year", genre: "$genres" }, total: { $sum: 1 } } },
  { $sort: { total: -1 } },
  { $group: { _id: "$_id.year", generoMasFrecuente: { $first: "$_id.genre" }, cantidad: { $first: "$total" } } },
  { $project: { _id: 0, year: "$_id", generoMasFrecuente: 1, cantidad: 1 } }
]);
```



```
/* 1 */
{
  "generoMasFrecuente" : "Western",
  "cantidad" : 251,
  "year" : 1948
},
/* 2 */
{
  "generoMasFrecuente" : "Drama",
  "cantidad" : 158,
  "year" : 1965
},
/* 3 */
{
  "generoMasFrecuente" : "Drama",
  "cantidad" : 178,
  "year" : 1955
},
/* 4 */
{
  "generoMasFrecuente" : "Comedy",
```

// 25 Sobre genres (nueva colección), encontrar los 5 años con mayor diversidad de géneros en películas, mostrando el año, el número total de géneros únicos y cuáles fueron esos géneros. Se filtra para descartar aquellos actores llamados "Undefined".

```
db.genres.aggregate([
  { $match: { genres: { $ne: "Undefined" } } }, // Filtrar géneros "Undefined"
  { $group: { _id: "$year", generos: { $addToSet: "$genres" } } },
  { $project: { _id: 1, totalGeneros: { $size: "$generos" }, generos: 1 } },
  { $sort: { totalGeneros: -1 } },
  { $limit: 5 }
]);
```

```

/* 1 */
{
  "_id" : 2013,
  "generos" : [
    "Crime",
    "Family",
    "Slasher",
    "Horror",
    "Mystery",
    "Historical",
    "Teen",
    "Adventure",
    "Satire",
    "Action",
    "Musical",
    "Found Footage",
    "Performance",
    "Documentary",
    "Drama",
    "Comedy",
    "Sports",
    "Superhero",
    "Science Fiction",
    "Biography",
    "Dance",
    "Romance",
    "Political",
    "Thriller",
    "War",
    "Animated",
    "Western",
    "Supernatural"
  ],
  "totalGeneros" : 31
},

```

```

    "Supernatural",
    "Noir",
    "Fantasy",
    "Erotic"
  ],
  "totalGeneros" : 31
},

```

```

/* 2 */
{
  "_id" : 2011,
  "generos" : [
    "Crime",
    "Family",
    "Horror",
    "Mystery",
    "Historical",
    "Teen",
    "Suspense",
  ],
  "totalGeneros" : 7
},

```

// 26. Encontrar a los 5 actores que han trabajado más en un solo género y muestra el número de películas realizadas de ese género.

```

db.genres.aggregate([
  { $match: { cast: { $ne: "Undefined" }, genres: { $ne: "Undefined" } } },
  { $group: { _id: { actor: "$cast", genre: "$genres" }, count: { $sum: 1 } } },
  { $group: { _id: "$_id.actor", GeneroTop: { $first: "$_id.genre" }, PelisTotales: { $max: "$count" } } },
  { $sort: { PelisTotales: -1 } },
  { $limit: 5 },
  { $project: { _id: 1, GeneroTop: 1, PelisTotales: 1 } }
]);

```

Aggregate (10) x Aggregate (11) x Aggregate (12) x Aggregate (13) x Result (14) x Result (15) x

genres 0.450 s 5 Docs

```
1  /* 1 */
2  {
3      "_id" : "Harold Lloyd",
4      "GeneroTop" : "Romance",
5      "PelisTotales" : 186
6  },
7
8  /* 2 */
9  {
10     "_id" : "Hoot Gibson",
11     "GeneroTop" : "Drama",
12     "PelisTotales" : 131
13 },
14
15 /* 3 */
16 {
17     "_id" : "Charles Starrett",
18     "GeneroTop" : "Crime",
19     "PelisTotales" : 103
20 },
21
22 /* 4 */
23 {
24     "_id" : "Johnny Mack Brown",
```